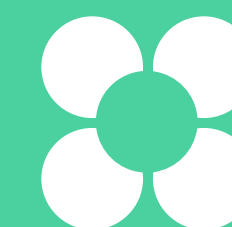


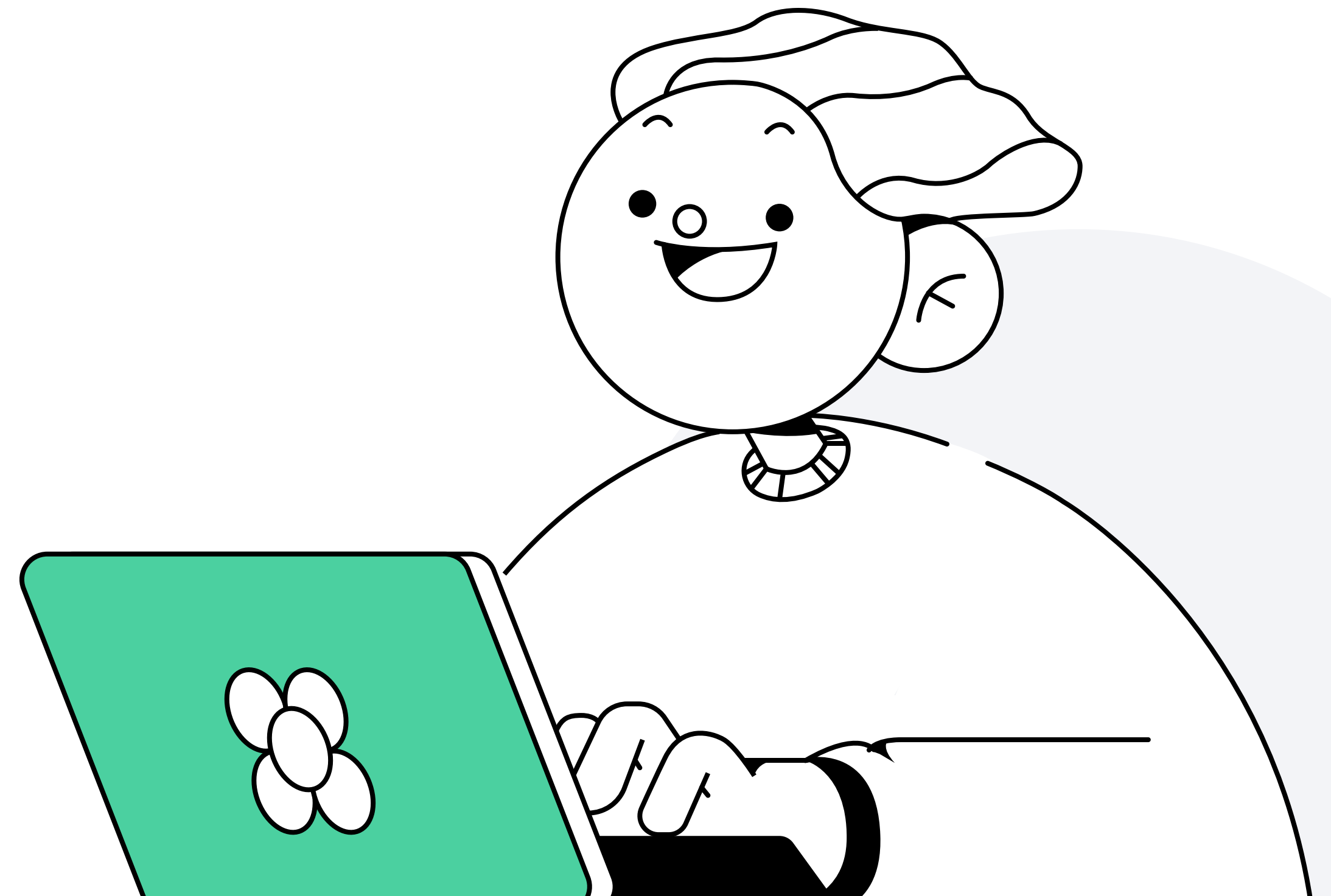
Жизненный цикл ПО

Алексей Метляков
Tech lead DevOps engineer в OpenWay



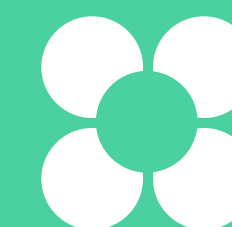
План занятия

- 1 Жизненный цикл ПО
- 2 Waterfall
- 3 Agile и Lean
- 4 Гибкие методологии



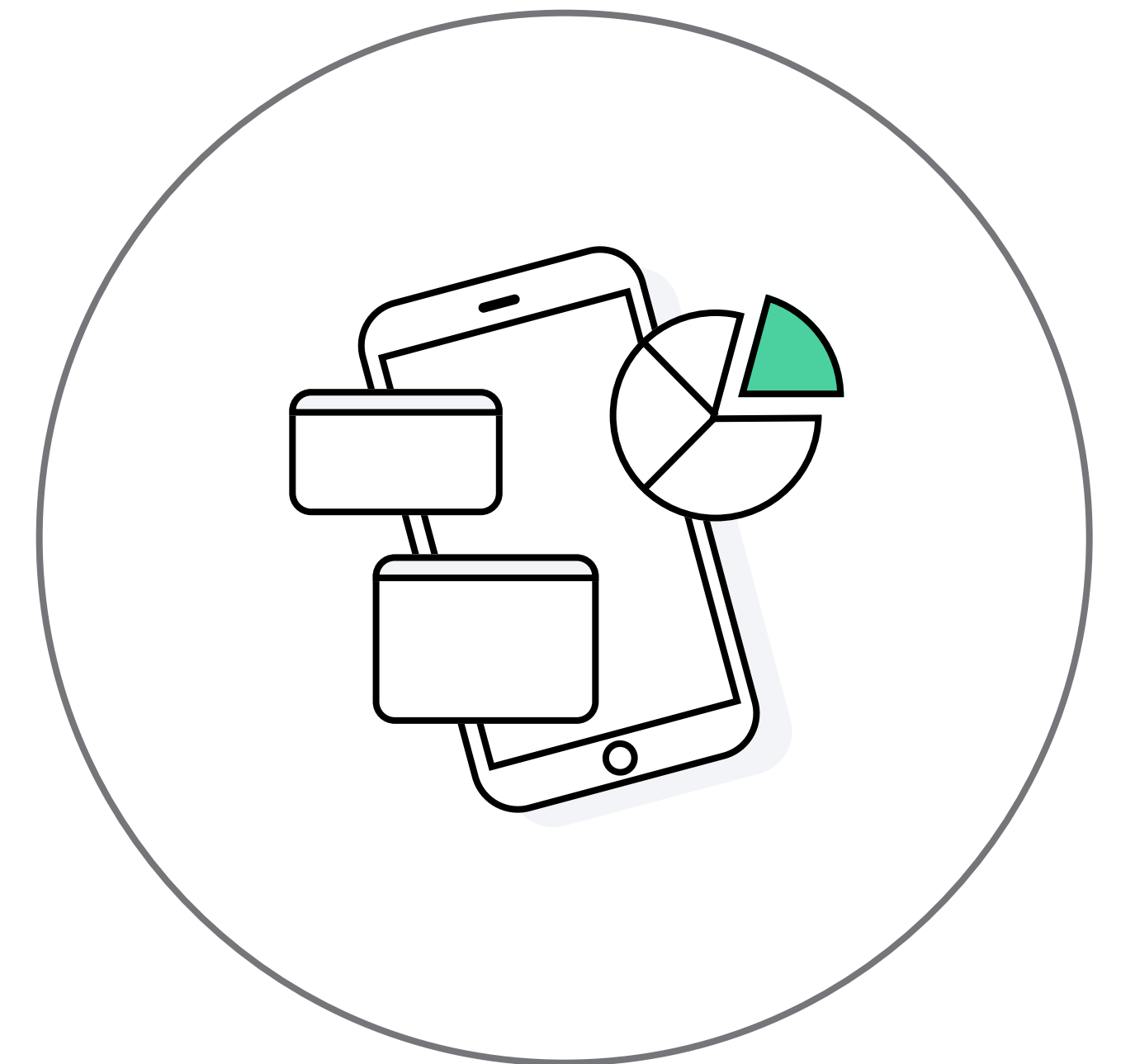
Жизненный цикл ПО

Алексей Метляков
Tech lead DevOps engineer в OpenWay



Что такое жизненный цикл ПО

Жизненный цикл ПО — ряд событий (этапов), происходящих с системой от её создания до дальнейшего использования и изъятия из эксплуатации



Зачем знать жизненный цикл ПО

Знать жизненный цикл ПО нужно, чтобы:

- понимать общую картину развития продукта
- определять стратегические и тактические цели команды
- лучше управлять продуктом
- обеспечить согласование требований заказчика

Из чего состоит жизненный цикл ПО

Жизненный цикл ПО состоит из следующих этапов:

- разработка
- тестирование
- внедрение
- сопровождение

Из чего состоит жизненный цикл ПО

Разработка состоит из следующих этапов:

- составление требований
- проектирование
- дизайн
- разработка

Из чего состоит жизненный цикл ПО

Тестирование состоит из следующих этапов:

- функциональное тестирование
- интеграционное тестирование
- нагрузочное тестирование
- показ

Из чего состоит жизненный цикл ПО

Внедрение состоит из следующих этапов:

- выделение инфраструктуры
- настройка рабочего окружения
- внедрение продукта
- тестирование работоспособности

Из чего состоит жизненный цикл ПО

Сопровождение состоит из следующих этапов:

- мониторинг работоспособности приложения
- работа с инцидентами
- вывод из эксплуатации

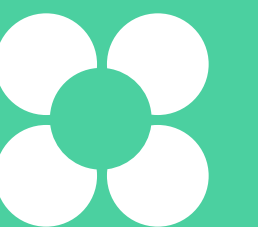
Из чего состоит жизненный цикл ПО

Жизненный цикл ПО состоит из следующих этапов:

Разработка	Тестирование	Внедрение	Сопровождение
Составление требований	Функциональное тестирование	Выделение инфраструктуры	Мониторинг работоспособности
Проектирование	Интеграционное тестирование	Настройка рабочего окружения	Работа с инцидентами
Дизайн	Нагрузочное тестирование	Внедрение продукта	Вывод из эксплуатации
Разработка	Показ	Тестирование работоспособности	

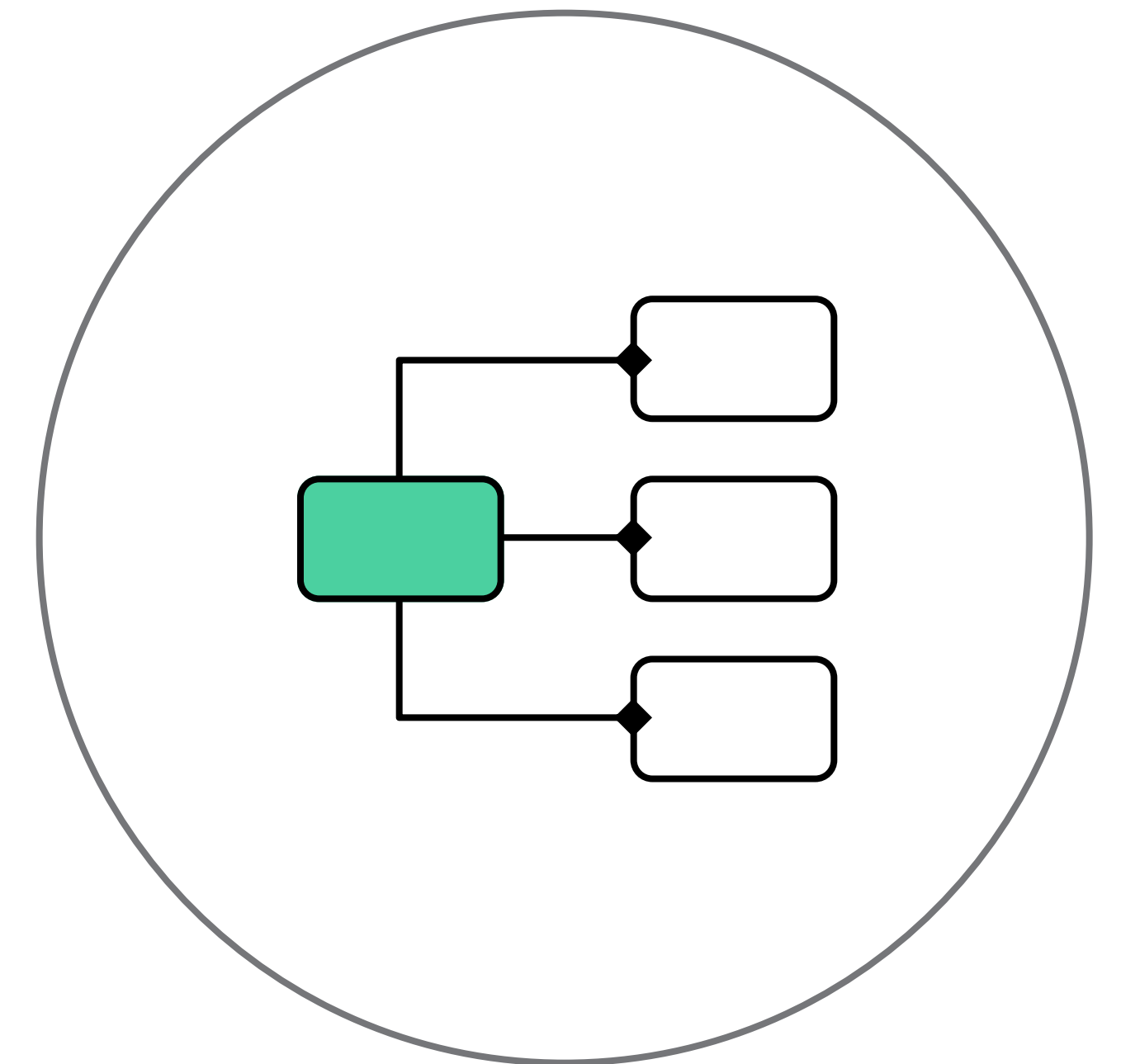
Waterfall

Алексей Метляков
Tech lead DevOps engineer в OpenWay



Waterfall

Waterfall – последовательная методология разработки и тестирования ПО. Концепция заключается в последовательном выполнении всех шагов, чтобы получить конечный продукт



Waterfall

Плюсы

- ⊕ стабильность и предсказуемость результата
- ⊕ простота отслеживания текущего состояния разработки
- ⊕ полноценная документация

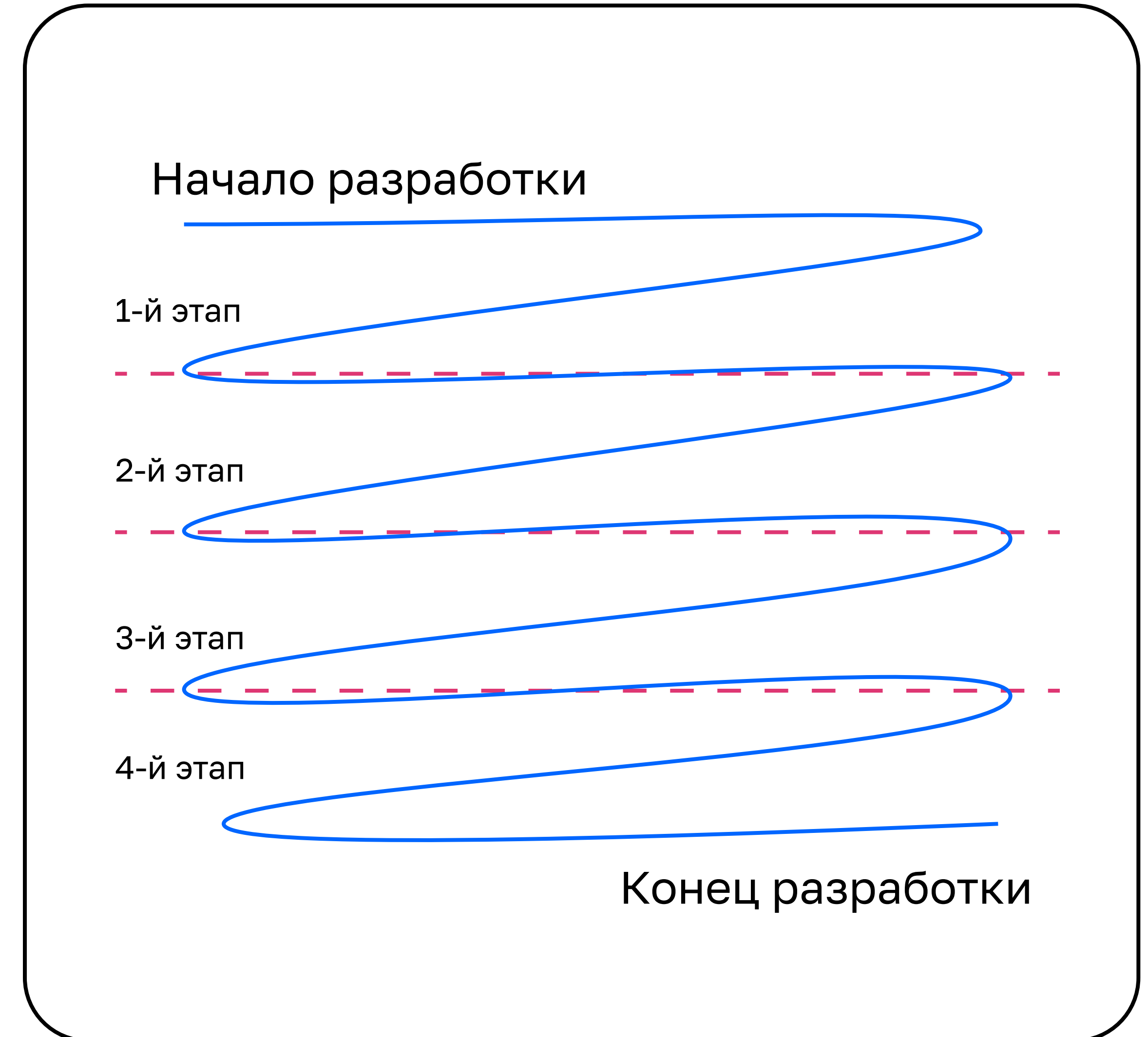
Минусы

- ⊖ медленная разработка
- ⊖ выявление ошибок разработки на поздних этапах

Waterfall

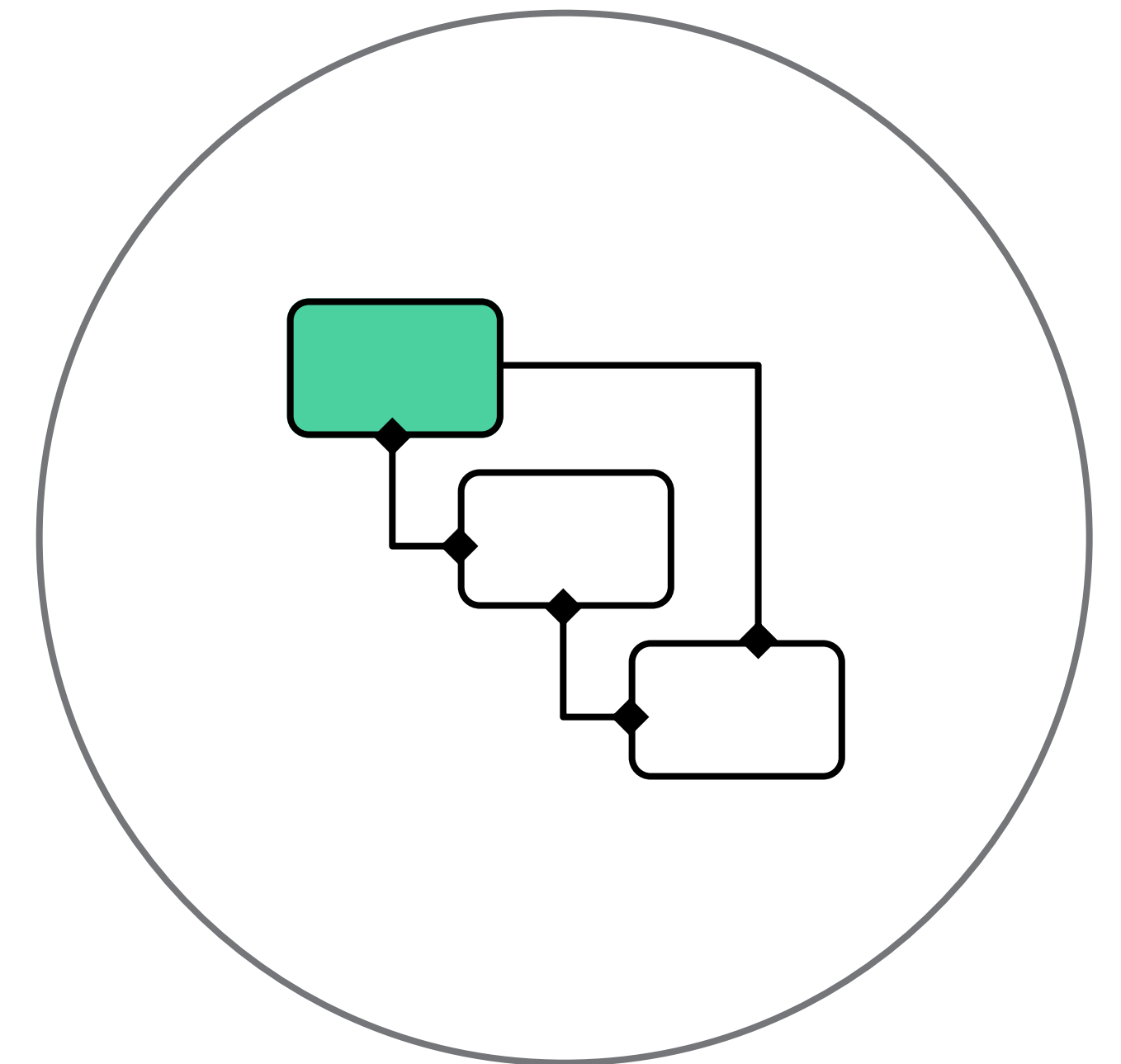
Спиральная модель — последовательная методология разработки и тестирования ПО. Концепция заключается в последовательном выполнении шагов, которые разделены на несколько этапов.

К плюсам добавляется возможность отслеживать промежуточный результат разработки в момент завершения очередного витка итерации



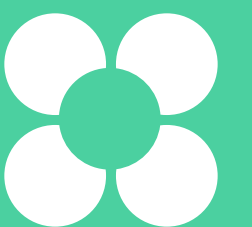
Waterfall

Инкрементная модель — последовательная методология разработки и тестирования ПО. Основная концепция Waterfall дополняется версионированием



Agile и Lean

Алексей Метляков
Tech lead DevOps engineer в OpenWay



Принципы Agile

Agile — набор из 12 принципов гибкой разработки ПО:

- Приоритет команды в удовлетворении заказчика
- Приветствуется постоянное изменение требований
- Рабочий продукт следует выпускать как можно чаще
- На протяжении всего проекта команда должна работать вместе
- Над проектом должны работать мотивированные профессионалы
- Важно непосредственное общение
- Работающий продукт — показатель прогресса
- Вся команда должна иметь возможность поддерживать постоянный рабочий ритм
- Постоянное внимание к совершенству и качеству повышает гибкость
- Простота необходима
- Лучшие требования рождаются у самоорганизующихся команд
- Команда должна систематически анализировать возможности улучшить эффективность работы и корректировать свой стиль

Достоинства Agile

На бумаге **Agile** выглядит как решение всех проблем:

- гибкий жизненный цикл ПО
- легко принимать изменения
- уменьшается бюрократический процесс в разработке
- быстро доставляются новые версии ПО
- качество кода улучшается без потерь в наращивании возможностей

Недостатки Agile

В жизни с **Agile** часто возникают проблемы:

- документация не успевает за кодовой базой проекта
- повышается сложность сопровождения продукта
- количество задач на разработку постоянно растёт
- большинство разработчиков занимается просмотром PR
- в маленьких командах на разработчиков ставят задачи тестирования
- времени создавать окружение автотестирования не хватает
- в рамках Agile у продукта гибкость требований влечёт за собой гибкость окружения

Lean

Исторически бережливое производство пришло из промышленности. Первая компания – Toyota.

Постулаты:

- устранение отходов
- расширение прав и возможностей работников
- уменьшение запасов
- повышение производительности

Фактически они стремились заключать контракты и собирать ровно столько машин, сколько у них заказали, не создавая сверх меры

Lean

Lean — принцип разработки, основанный на бережливом производстве, смысл которого в устранении помех производству. Устранение происходит в два этапа:

- **анализ** — проводят анализ всех процессов в алгоритме работы команды, определяют уязвимости, траты ресурсов и времени, «бутылочные горлышки»
- **внесение изменений** — предлагаются альтернативы процессам алгоритма. Альтернатива не обязательно должна быть инновационной — она должна просто улучшить конечный результат. Все альтернативы считаются возможными и исследуются на практике. На основе практического подхода выбирают, какая из альтернатив лучшая, и используют в постоянном алгоритме разработки

Концепции Lean в управлении «отходами»

Муда, мура, мури – три концепции управления «отходами» разработки.

Муда – отходы, иными словами, последствия ошибок.

Примеры:

- бесполезные данные, которые отправляются из приложения в БД
- несоответствие ТЗ
- безразличие к рабочему процессу
- непрошенная самодеятельность в готовом проекте
- накопленные запросы без движения

Концепции Lean в управлении «отходами»

Мура — причины появления муда (мусора). Например, неравномерная по времени нагрузка на команду.

Мури — необоснованные сложности в работе (нецелесообразность).

Примеры:

- разработчик занимается тестированием
- у команды нет необходимых инструментов для разработки
- плохо поставленные задачи в ТЗ
- чтобы выполнить задачу, требуется множество звонков и согласований

Основной смысл Lean — убрать все три категории, при этом нужно бороться с причиной, а не следствием

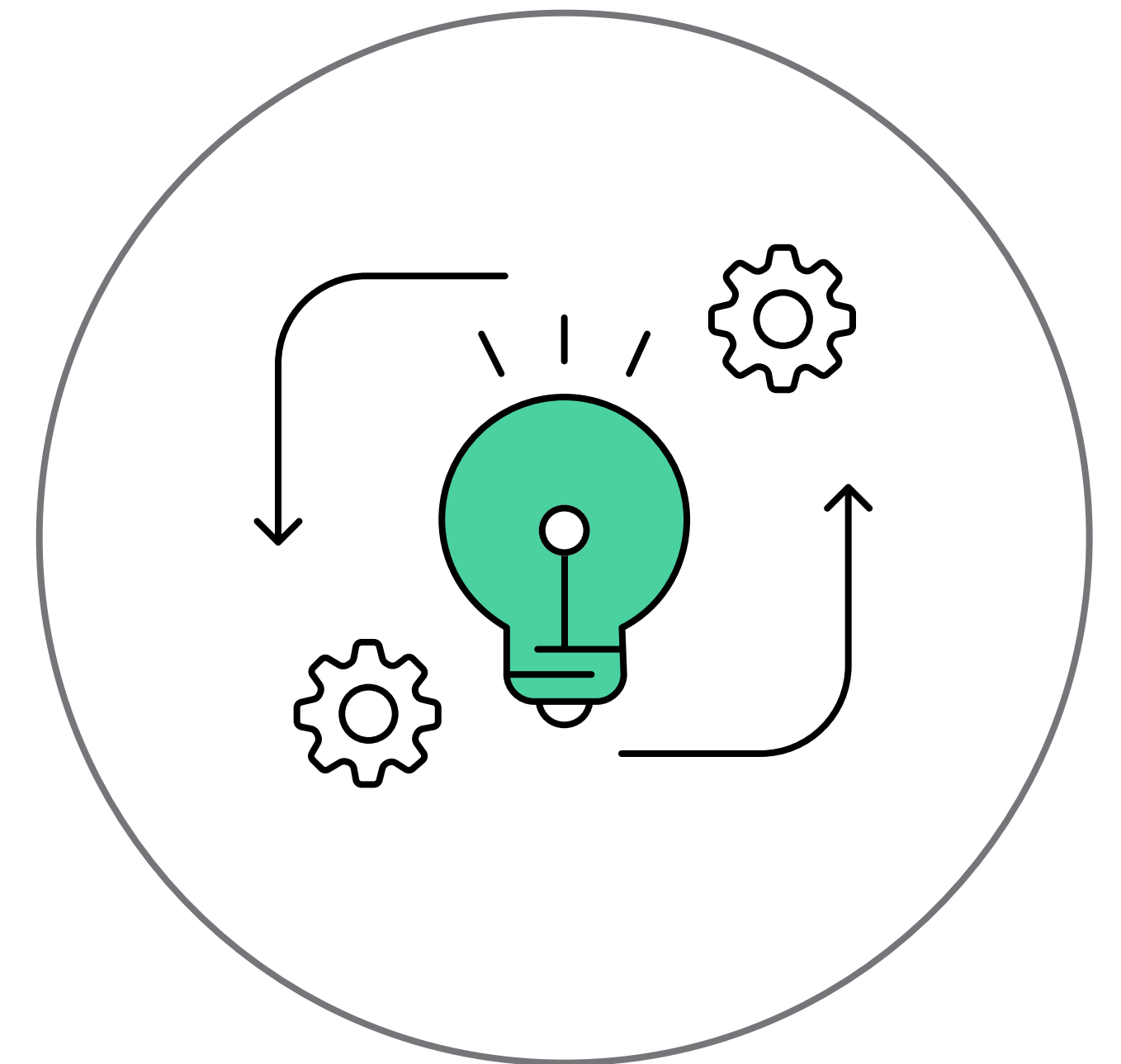
Принципы Lean

Основные принципы **Lean**:

- ликвидировать мусор
- создавать только то, что нужно
- все задачи должны исходить из желаний потребителя
- делать правильно с первого раза
- расширять возможности команды
- создать культуру постоянного совершенствования

Кайдзен в Lean

Кайдзен — (кай — изменения, дзен — хорошо) метод, позволяющий команде непрерывно предлагать и тестировать идеи по улучшению своей работы, работы всей команды и программного продукта



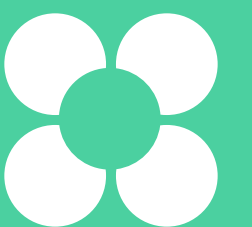
Кайдзен в Lean

Пять характеристик кайдзен:

- равноправное **взаимодействие** всех участников команд и прямая **коммуникация** между ними
- индивидуальная **дисциплина**
- здоровое **моральное состояние** всей команды и каждого участника
- кружки качества
- **предложения** по улучшению всего в рамках работы

Гибкие методологии

Алексей Метляков
Tech lead DevOps engineer в OpenWay



Глоссарий

- **Product manager** — тот, кто владеет продуктом
- **Project manager** — тот, кто управляет кастомизацией продукта для одного клиента
- **Team lead (тимлид)** — программист, управляющий командой
- **QA** — тестировщик
- **Front** — разработчик интерфейсов
- **Back** — разработчик «подкапотного» кода продукта
- **Ops** — сопровождение продукта
- **Delivery** — тот, кто отвечает за доставку продукта до клиентов
- **Bug (баг)** — найденное отклонение от ожидаемого поведения продукта
- **Task** — задача на изменение продукта
- **Feature** — задача на разработку новых функций продукта
- **Epic** — верхнеуровневое описание какой-то общей цели
- **Stories** — описание результата, которого хочется достичь
- **Workflow** — движение задач разных видов по статусам
- **Backlog** — общий пул задач

Kanban

Kanban – один из способов управлять потоком работы, разделяя все задачи по столбцам статусов, в рамках которых команда понимает, какие задачи сейчас:

- находятся в разработке
- нужно выполнить в ближайшее время
- уже сделаны
- НЕВОЗМОЖНО ВЫПОЛНИТЬ

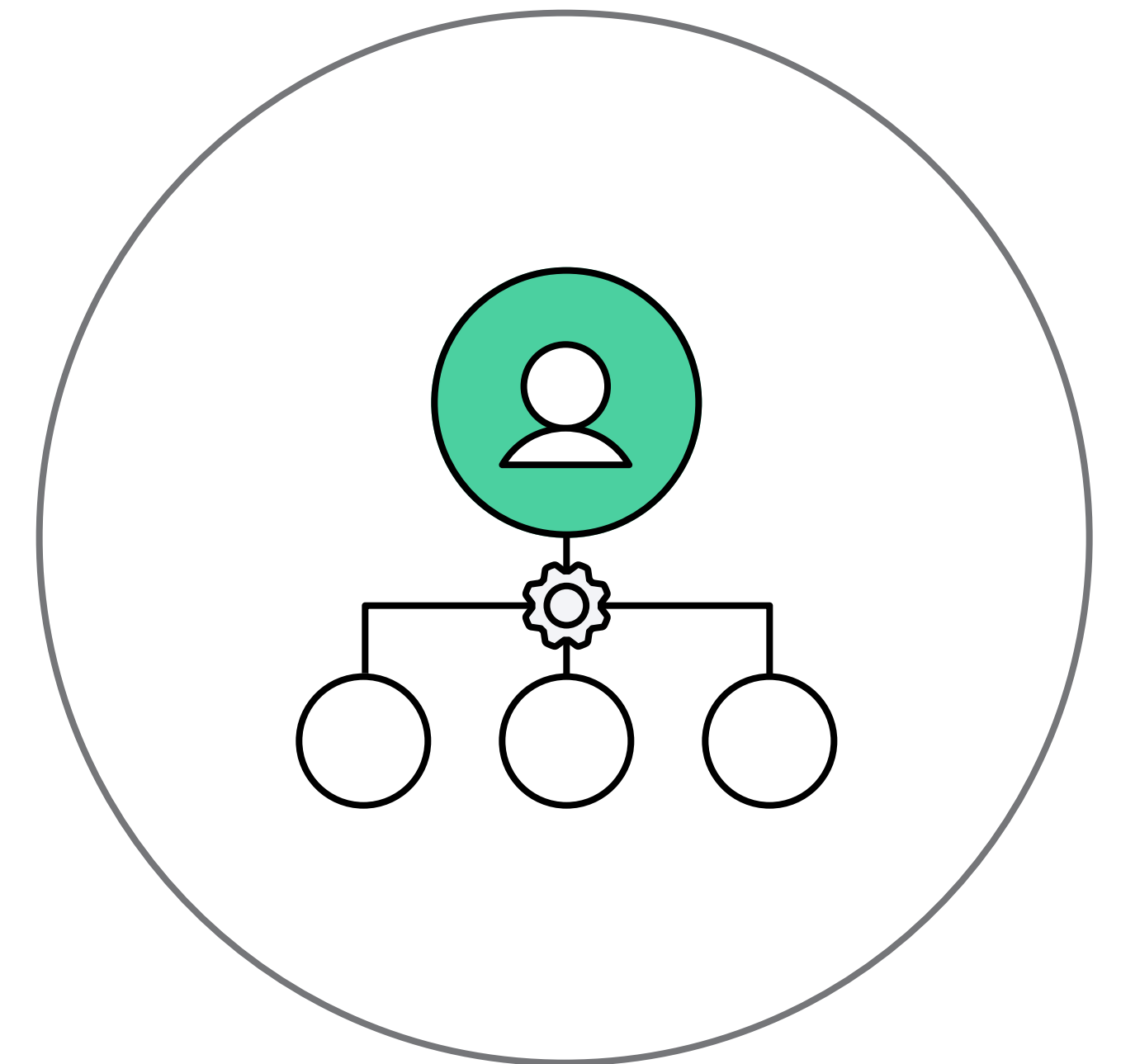
Kanban

При использовании **Kanban** могут возникнуть трудности:

- неявный переход между статусами
- неуправляемый поток задач
- централизованное управление
- неясный план
- пренебрежение качеством

Scrum

Scrum – способ управлять рабочим процессом команды, распределяя задачи на спринты (sprint)



Scrum

Scrum – способ управлять рабочим процессом команды, распределяя задачи на спринты (sprint)

Спринт – промежуток времени, за который команда гарантирует выполнение определённого набора задач

Scrum

Обычно **спринт** завершается созданием релиза продукта:

- задачи для спринта команда выбирает на планировании
- задачи выбирают, исходя из сложности
- сложность вычисляют опытным путём
- первое завершение спринта у новой команды обычно не соответствует ожиданиям

Scrum

При использовании Scrum могут возникнуть трудности:

- задачи накапливаются в бэклоге
- задачи переносятся из одного спринта в следующий и не выполняются
- график выгорания задач стремится к нулю